Spring Framework Deep-Dive

IoC and Dependency Injection Fundamentals

Chapter 01 -- From Tight Coupling to Container-Managed Beans

A. Problem Without IoC	Tight coupling, new everywhere
B. Inversion of Control	Who controls object creation?
C. Constructor Injection	Preferred, immutable, testable
D. Setter Injection	Optional dependencies
E. Field Injection (@Autowired)	AutowiredAnnotationBeanPostProcessor
F. Stereotype Annotations	@Component @Service @Repository @Controller
G. @Qualifier and @Primary	Resolving ambiguity
H. @Configuration and @Bean	ConfigurationClassPostProcessor + CGLIB

A The Problem Without IoC

Tight coupling, new keyword everywhere -- why this breaks

Step 1: Plain English -- What Is the Problem?

When a class uses the **new** keyword to create its own dependencies, it becomes tightly coupled to those concrete implementations. The class knows exactly which class it depends on, which version, and how to construct it. This violates the Single Responsibility Principle -- the class does its job AND manages wiring. It makes unit testing nearly impossible because you cannot swap in a mock, and creates cascading recompilation whenever a dependency changes.

The core issue: the class that USES a dependency should not be responsible for CREATING it. These are two entirely separate concerns.

Step 2: Real-World Analogy

> Analogy: A chef who, instead of being handed ingredients, must personally drive to the farm for vegetables, the butcher for meat, and the mill for flour every time they cook. That chef is tightly coupled to specific suppliers. A real kitchen has a separate supply chain (IoC container) that delivers exactly what is needed. The chef declares: 'I need flour, eggs, butter' -- and they appear.

Step 3: Minimal Working Code -- The Problem

```
// Tightly coupled: OrderController creates everything itself
public class OrderController {
    private OrderService orderService;

    public OrderController() {
        // BAD: controller hard-codes concrete repo class
        OrderRepository repo = new JdbcOrderRepository();
        this.orderService    = new OrderService(repo);
    }

    public void placeOrder(Order o) {
        orderService.process(o);
    }
}
// To unit-test OrderController you must instantiate
// JdbcOrderRepository -- which needs a real database.
// There is no way to inject a MockOrderRepository.
```

Step 4: Diagram

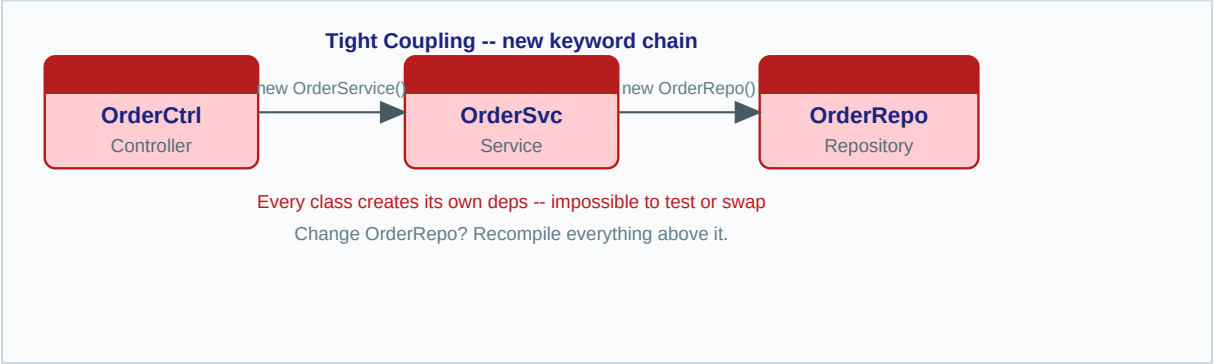

Fig A: Tight coupling -- each class hardwires its dependency graph via new

Step 5: Behind the Scenes -- Design Pattern Violated

The design principle violated here is the **Dependency Inversion Principle** (the D in SOLID): high-level modules should not depend on low-level modules; both should depend on abstractions. When you use **new ConcreteClass()**, you bind to the implementation, not the interface. There are no Spring classes involved yet -- that is exactly the point. Without a container, the developer manually manages the entire object graph.

A secondary problem is the **Service Locator anti-pattern**: code that calls SomeFactory.getInstance() is still tightly coupled, just one layer removed. Neither approach gives lifecycle management, circular dependency detection, or scope control.

Step 6: Realistic Practical Example

```
// Real-world pain: switching from JDBC to JPA means touching
// EVERY class that calls new JdbcOrderRepository()
public class ReportService {
    // Hard-coded. Cannot use InMemoryRepo in tests.
    private OrderRepository repo = new JdbcOrderRepository(
        "jdbc:mysql://prod-server/orders", "root", "secret"
    );

    public List generateMonthlyReport() {
        return repo.findByMonth(LocalDate.now());
    }
}
// Test for ReportService requires a running MySQL instance.
// Any password rotation breaks the build.
```

Step 7: Common Mistakes and Misconceptions

Misconception: 'I use a factory method, so I am not tightly coupled.' Wrong. If OrderFactory.create() returns a concrete JdbcOrderRepository, the caller is still coupled to that decision. True decoupling requires the dependency to be INJECTED from outside the class, not obtained from a known factory.

Misconception: 'Static utility classes are fine for shared dependencies.' Wrong. Static state cannot be overridden in tests, cannot be proxied by Spring AOP, and introduces hidden global state into your application. Spring beans solve all three.

B Inversion of Control

ApplicationContext, BeanDefinition, DefaultListableBeanFactory

Step 1: Plain English -- What Is IoC?

Inversion of Control is the design principle where the framework calls YOUR code, rather than your code calling the framework. For object creation specifically: instead of your code deciding when and how to instantiate dependencies, you declare WHAT you need, and the Spring container decides WHEN to create it, HOW to wire it, and WHEN to destroy it.

IoC is a broad principle. **Dependency Injection** is the most common mechanism implementing IoC in Spring. The container reads configuration (annotations or XML), builds a complete map of all beans and their relationships, then creates and wires everything before handing any object to your code.

Step 2: Real-World Analogy

> Analogy: A staffing agency (IoC container) holds a roster of workers (beans). When your company (application) needs a Java developer, you tell the agency your requirements (interface/type). The agency finds the right person, handles the contract (lifecycle), provides benefits (AOP, transactions), and delivers them ready to work. You never interview candidates yourself -- control is inverted to the agency.

Step 3: Minimal Working Code

```
@Configuration
@ComponentScan("com.example")
public class AppConfig { }

// Main class -- asks the container, never calls new for services
public class Application {
    public static void main(String[] args) {
        ApplicationContext ctx =
            new AnnotationConfigApplicationContext(AppConfig.class);
        // Container created OrderService and wired all its deps
        OrderService svc = ctx.getBean(OrderService.class);
        svc.process(new Order());
    }
}
```

Step 4: Diagram

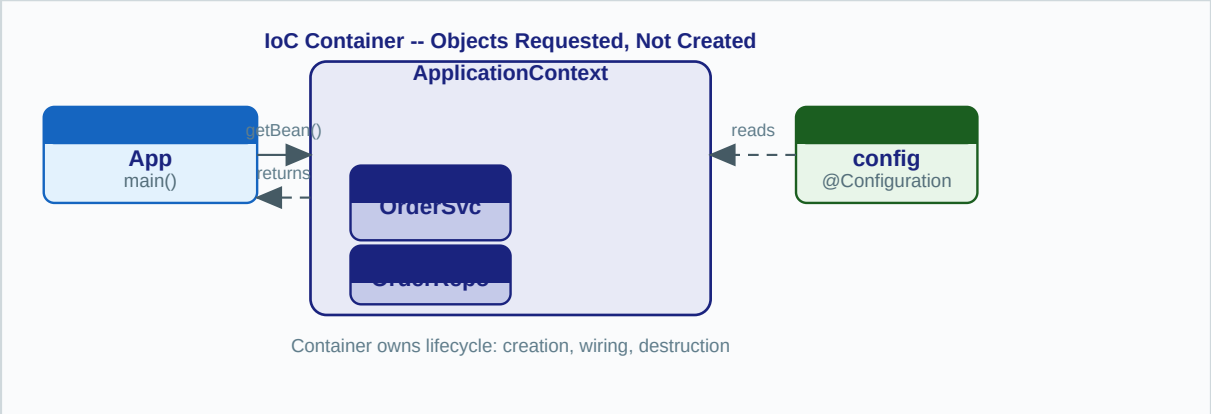

Fig B: IoC Container manages object creation; application merely requests beans

Step 5: Behind the Scenes -- Spring Internals

The central class is **DefaultListableBeanFactory**, which implements both **BeanFactory** and **BeanDefinitionRegistry**. The startup sequence:

* **BeanDefinitionReader** parses @Configuration classes or XML and creates **BeanDefinition** objects. Each BeanDefinition stores: bean class name, scope (singleton/prototype), lazy-init flag, constructor argument values, property values, init-method name, and destroy-method name.

* **BeanDefinitionRegistry.registerBeanDefinition(name, def)** stores each BeanDefinition in a ConcurrentHashMap keyed by bean name inside DefaultListableBeanFactory.

* When a bean is first requested, **AbstractBeanFactory.getBean()** checks the singleton cache. If absent, it delegates to **AbstractAutowireCapableBeanFactory.createBean()**.

* createBean() calls instantiateBean() (reflection via Constructor.newInstance()), runs all registered **BeanPostProcessor** implementations (populateBean, initializeBean), then registers the fully initialized bean in the singleton cache.

The design pattern is **Template Method**: AbstractBeanFactory defines the algorithm (getBean), and subclasses override specific steps (createBean). IoC itself follows the Hollywood Principle: 'Don't call us, we'll call you.'

Step 6: Realistic Practical Example

```
// Spring Boot -- IoC wires the entire object graph automatically
@SpringBootApplication  // enables @ComponentScan + @Configuration
public class EcommerceApp {
    public static void main(String[] args) {
        // This single line bootstraps DefaultListableBeanFactory,
        // scans com.example.**, registers ~200+ BeanDefinitions,
        // creates all singleton beans, and fires ApplicationListeners.
        // Your code contributes @Service/@Repository/@Controller classes.
        SpringApplication.run(EcommerceApp.class, args);
    }
}
```

Step 7: Common Mistakes and Misconceptions

Misconception: 'ApplicationContext and BeanFactory are the same.' BeanFactory is the base interface (lazy bean creation). ApplicationContext extends it and adds: eager singleton initialization, event publishing, i18n MessageSource, and environment abstraction. Always use ApplicationContext in real applications.

Misconception: 'IoC means the framework controls ALL my code.' IoC applies specifically to object lifecycle and wiring. Your business logic runs exactly as you write it. The container only controls instantiation, dependency resolution, and destruction.

C

Constructor Injection

Reflection, immutability, testability -- the preferred approach

Step 1: Plain English -- What Is Constructor Injection?

Constructor injection means Spring passes all required dependencies as arguments to the class constructor when creating the bean. Your class declares what it needs in its constructor signature, and the container fulfills those requirements automatically. The class never calls **new** for its own dependencies.

Constructor injection is the **recommended approach** in Spring 4.3+. Since Spring 4.3, if a class has exactly one constructor, @Autowired is not even required -- Spring infers it automatically.

Step 2: Real-World Analogy

> Analogy: A surgeon needs a scalpel, gloves, and anesthesiologist before operating. These are non-optional: the surgery cannot begin without them. When the surgeon enters the OR (constructor is called), a scrub nurse has already laid out every required instrument (Spring injects dependencies). The operation cannot START without everything in place -- exactly like constructor injection with required deps.

Step 3: Minimal Working Code

```
@Service
public class OrderService {
    // final = immutable after construction; proves no circular dep
    private final OrderRepository orderRepository;
    private final EmailService emailService;

    // Spring 4.3+: no @Autowired needed with single constructor
    public OrderService(OrderRepository orderRepository,
                        EmailService emailService) {
        this.orderRepository = orderRepository;
        this.emailService    = emailService;
    }

    public void placeOrder(Order order) {
        orderRepository.save(order);
        emailService.sendConfirmation(order);
    }
}
```

Step 4: Diagram

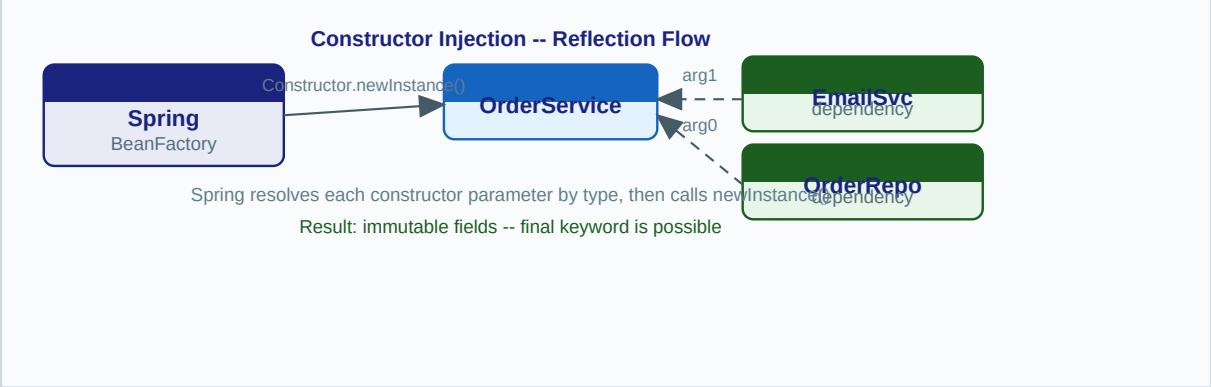

Fig C: Spring uses Constructor.newInstance() -- all args resolved before the call

Step 5: Behind the Scenes -- Reflection Deep Dive

Spring uses **java.lang.reflect.Constructor.newInstance(Object... args)** to create beans. The exact sequence inside **AbstractAutowireCapableBeanFactory**:

* **determineConstructorsFromBeanPostProcessors()** -- AutowiredAnnotationBeanPostProcessor inspects all constructors for @Autowired. If a single public constructor exists, it is selected automatically without @Autowired.

* **resolveConstructorArguments()** -- For each constructor parameter, Spring calls **DefaultListableBeanFactory.resolveDependency()** which looks up a matching bean by type (and optionally name). This is where @Qualifier is evaluated.

* **instantiateBean()** -- The resolved arguments array is passed to **BeanUtils.instantiateClass(ctor, args)** which calls **ctor.newInstance(args)**.

* The result is a raw object. BeanPostProcessors then run populateBean and initializeBean phases to complete initialization.

Why constructor injection is preferred: (1) **Immutability** -- fields can be **final**, guaranteed set exactly once. (2) **Testability** -- unit tests call the constructor directly with mocks, no Spring context needed. (3) **Fail-fast** -- missing dependencies cause BeanCreationException at startup, not NullPointerException at runtime. (4) **Circular dep detection** -- Spring detects A-requires-B-requires-A with constructors and throws immediately.

Step 6: Realistic Practical Example

```
@RestController
public class PaymentController {
    private final PaymentService paymentService;
    private final AuditLogger auditLogger;

    // Lombok @RequiredArgsConstructor can generate this automatically
    public PaymentController(PaymentService paymentService,
                             AuditLogger auditLogger) {
        this.paymentService = paymentService;
        this.auditLogger    = auditLogger;
    }

    @PostMapping("/pay")
    public ResponseEntity pay(@RequestBody PaymentRequest req) {
        auditLogger.log("Payment attempt: " + req.getAmount());
        return ResponseEntity.ok(paymentService.process(req));
    }
}
```

Step 7: Common Mistakes and Misconceptions

> Mistake: Using @Autowired on a constructor when there is only one constructor. Since Spring 4.3, this annotation is redundant and adds visual noise. Omit it. If you have multiple constructors, annotate the one Spring should use.

> Mistake: Constructor injection with circular dependencies. If A requires B in its constructor and B requires A in its constructor, Spring throws BeanCurrentlyInCreationException. Fix by redesigning to break the cycle, or use setter injection for one side (Spring can resolve setter-based circular deps at init time).

D

Setter Injection

Optional dependencies, two-phase instantiation, circular dep resolution

Step 1: Plain English -- What Is Setter Injection?

Setter injection means Spring first creates the bean using its no-arg constructor, then calls annotated setter methods to provide dependencies. Each setter receives one dependency. This is a two-phase process: instantiation, then population.

Setter injection is best for **optional** dependencies -- ones that have a sensible default and are not strictly required for the bean to function. It also allows re-injection after construction, which is useful in certain testing scenarios.

Step 2: Real-World Analogy

> Analogy: Building a car on an assembly line. First the chassis rolls off (object created with no-arg constructor -- bare frame exists). Then stations add components: engine at station 2, wheels at station 3, seats at station 4. Each addition is a setter call. The car exists in a partially built state between stations -- that is the risk of setter injection if initialization logic runs too early.

Step 3: Minimal Working Code

```
@Service
public class NotificationService {
    private EmailSender emailSender;    // required
    private SmsSender   smsSender;      // optional

    @Autowired  // required=true by default; throws if missing
    public void setEmailSender(EmailSender emailSender) {
        this.emailSender = emailSender;
    }

    @Autowired(required = false)  // app works without SMS capability
    public void setSmsSender(SmsSender smsSender) {
        this.smsSender = smsSender;
    }

    public void notify(String msg) {
        emailSender.send(msg);
        if (smsSender != null) smsSender.send(msg);
    }
}
```

Step 4: Diagram

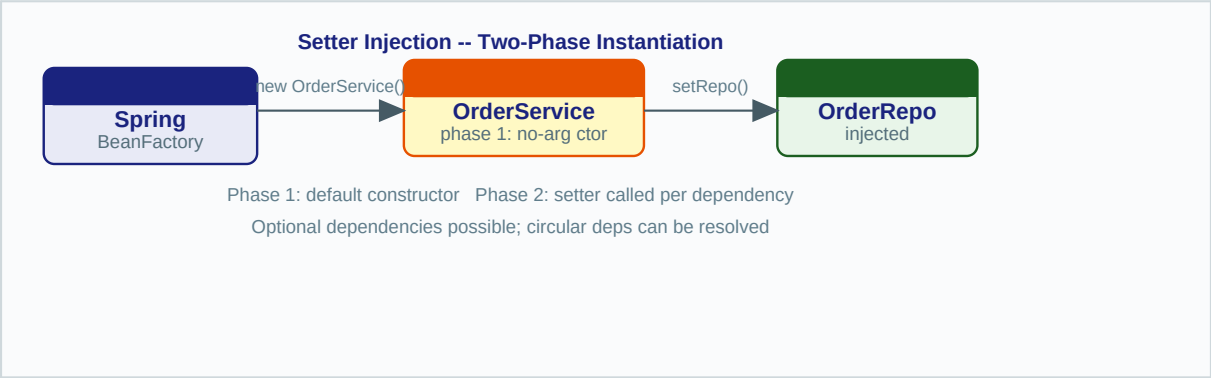

Fig D: Two-phase instantiation -- bean exists before dependencies are injected

Step 5: Behind the Scenes

Inside **AbstractAutowireCapableBeanFactory.populateBean()**: after the bean is instantiated, Spring's **AutowiredAnnotationBeanPostProcessor.postProcessProperties()** scans all methods annotated with @Autowired. For each, it calls resolveDependency() and then invokes the method via reflection: **method.invoke(beanInstance, resolvedValue)**.

Setter injection CAN resolve circular dependencies because Spring can create both beans first (bare objects via no-arg constructors) and then populate each other's references in the population phase. This is why circular dependencies work with setters but fail with constructors. However, this means the bean is in an incomplete state during part of its creation -- a hazard if any initializer method runs before all setters fire.

Step 6: Realistic Practical Example

```
@Component
public class ReportGenerator {
    private DataSource  dataSource;     // required
    private CacheManager cacheManager;  // optional optimization

    @Autowired
    public void setDataSource(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Autowired(required = false)
    public void setCacheManager(CacheManager cacheManager) {
        this.cacheManager = cacheManager;
    }

    public Report generate(ReportRequest req) {
        if (cacheManager != null) {
            Report cached = cacheManager.get(req.getId());
            if (cached != null) return cached;
        }
        return buildReport(req);
    }
}
```

Step 7: Common Mistakes and Misconceptions

> Mistake: Using setter injection for required dependencies. Setter injection is the right tool only for optional deps. For anything mandatory, use constructor injection so the application fails fast at startup rather than with a NullPointerException the first time the method is actually called.

Misconception: 'Setter injection is deprecated in Spring 6.' It is not deprecated. The official Spring recommendation since 2016: constructor injection for required deps, setter injection for optional deps. Both remain fully supported.

E Field Injection -- @Autowired

AutowiredAnnotationBeanPostProcessor internals, Field.setAccessible

Step 1: Plain English -- What Is Field Injection?

Field injection means you place @Autowired directly on an instance field. Spring uses reflection to set the field's value after bean instantiation, bypassing any setter method or constructor parameter. It is the most concise form but has the most drawbacks.

Spring must use **Field.setAccessible(true)** to inject even private fields. This works at runtime but the Java module system (Java 9+ strong encapsulation) may block it unless the module opens its packages to Spring.

Step 2: Real-World Analogy

> Analogy: A delivery courier slips packages through your mail slot while you sleep. You wake up and the items are there, but you have no record of receiving them, no control over when they arrived, and cannot verify they came from the right sender. Field injection is similar: dependencies appear magically, but the mechanism is opaque and bypasses your class's own initialization logic.

Step 3: Minimal Working Code

```
@Service
public class UserService {
    // Field injection -- concise but hides the dependency contract
    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    public void registerUser(String username, String password) {
        String hashed = passwordEncoder.encode(password);
        userRepository.save(new User(username, hashed));
    }
}
// Unit test CANNOT call: new UserService()
// Fields will be null without Spring or Mockito @InjectMocks.
```

Step 4: Diagram

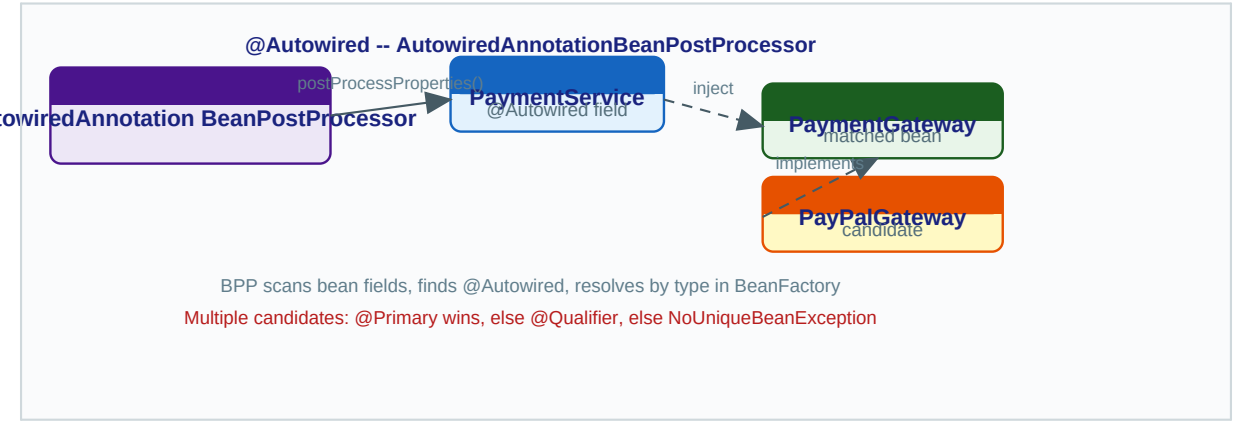

Fig E: AutowiredAnnotationBeanPostProcessor resolves @Autowired fields via reflection

Step 5: Behind the Scenes -- AutowiredAnnotationBeanPostProcessor

AutowiredAnnotationBeanPostProcessor (AABPP) is a BeanPostProcessor registered automatically with @ComponentScan or @EnableAutoConfiguration. Its key method: **postProcessProperties(PropertyValues pvs, Object bean, String beanName)**.

* **findAutowiringMetadata()** -- For a given bean class, AABPP scans all declared fields and methods looking for @Autowired, @Value, @Inject. Results are cached in a ConcurrentHashMap by class name for performance.

* For each @Autowired field, it creates an **AutowiredFieldElement** which calls **beanFactory.resolveDependency(DependencyDescriptor, beanName)**.

* **resolveDependency()** checks: (1) the field's type, (2) any @Qualifier annotation, (3) whether the field is a Collection or Map (inject all matching beans). If multiple beans match without @Qualifier or @Primary: NoUniqueBeanDefinitionException.

* Finally: **field.setAccessible(true); field.set(bean, resolvedValue)**.

Step 6: Realistic Practical Example

```
// Acceptable use of field injection: @SpringBootTest test classes
// Spring manages the full context here anyway.
@SpringBootTest
class OrderServiceIntegrationTest {
    @Autowired
    private OrderService orderService;

    @Autowired
    private OrderRepository orderRepository;

    @Test
    void shouldPersistOrder() {
        Order order = new Order("PROD-1", 2);
        orderService.placeOrder(order);
        assertThat(orderRepository.findAll()).hasSize(1);
    }
}
```

Step 7: Common Mistakes and Misconceptions

Major drawback: Field-injected beans cannot be unit-tested without a Spring context or Mockito's @InjectMocks. With constructor injection, a test just calls new MyClass(mock). With field injection, there is no public API to provide the dependency.

Major drawback: Hidden dependencies. A class with 8 @Autowired fields looks lightweight but secretly has 8 dependencies, silently violating Single Responsibility. Constructor injection makes an 8-dependency class obvious (8-argument constructor), naturally discouraging over-injection through the pain of writing the constructor.

Note: IntelliJ IDEA and Spring's own documentation warn against field injection in production classes. It is not deprecated but is discouraged. Restrict it to test classes where Spring manages the full context anyway.

F @Component, @Service, @Repository, @Controller

Differences, ClassPathBeanDefinitionScanner internals, when to use each

Step 1: Plain English -- What Are Stereotype Annotations?

All four annotations tell Spring: 'This class is a bean -- manage it.' They all trigger component scanning and result in a BeanDefinition being registered. The functional difference is minimal at runtime, but the **semantic** difference is significant: they communicate the ROLE of the class in application architecture.

@Service, @Repository, and @Controller are all meta-annotated with @Component. At their core they ARE @Component -- just with an additional label. That label activates additional framework behavior in specific cases.

Step 2: Real-World Analogy

> Analogy: All hospital staff are 'employees' (@Component). But labeling someone a 'Surgeon' (@Service), 'Lab Technician' (@Repository), or 'Receptionist' (@Controller) tells the hospital system what additional responsibilities apply. A Surgeon gets operating room access; a Lab Technician gets diagnostic equipment. Same base role, specialized behavior and visibility.

Step 3: Minimal Working Code

```
// @Controller: marks as MVC request handler (activates Spring MVC machinery)
@Controller
public class ProductController { ... }

// @Service: business logic layer. No extra Spring behavior by default.
// Communicates architectural role clearly to developers.
@Service
public class ProductService { ... }

// @Repository: data access layer. Spring wraps checked SQL/JPA exceptions
// in DataAccessException hierarchy via exception translation proxy.
@Repository
public class JpaProductRepository implements ProductRepository { ... }

// @Component: generic. Use when none of the above semantic labels fit.
@Component
public class CsvParser { ... }
```

Step 4: Diagram

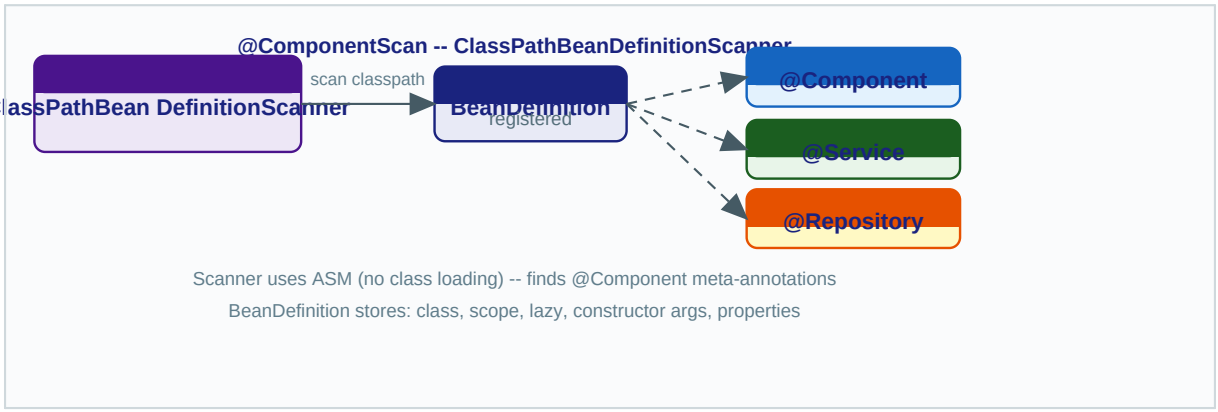

Fig F: ClassPathBeanDefinitionScanner finds annotated classes, registers BeanDefinitions

Step 5: Behind the Scenes -- ClassPathBeanDefinitionScanner

When you use @ComponentScan(basePackages='com.example'), Spring creates a **ClassPathBeanDefinitionScanner**:

* **scan(basePackages)** uses PathMatchingResourcePatternResolver to find all .class files under the package. It reads class metadata using the **ASM bytecode library** -- NOT actual class loading. This avoids triggering static initializers prematurely.

* **isCandidateComponent(MetadataReader)** checks if the class bears @Component or any annotation that is itself meta-annotated with @Component (like @Service, @Repository).

* For each candidate, a **ScannedGenericBeanDefinition** is created from class metadata and registered with a generated name (default: simple class name, first letter lowercase).

* A BeanDefinition stores: beanClassName, scope, lazyInit, primary, dependsOn, qualifiers, constructorArgumentValues, propertyValues, initMethodName, destroyMethodName.

The extra behavior of @Repository: **PersistenceExceptionTranslationPostProcessor** wraps any @Repository-annotated bean in a proxy that translates JPA/Hibernate exceptions into Spring's **DataAccessException** hierarchy. This is the ONLY behavioral difference at runtime -- @Service and plain @Component are identical.

Step 6: Realistic Practical Example

```
// Correct layering for an e-commerce module
@RestController                      // = @Controller + @ResponseBody
@RequestMapping("/api/orders")
public class OrderController {
    private final OrderService orderService;
    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }
}

@Service
@Transactional   // services typically own transactions
public class OrderService {
    private final OrderRepository repo;
    public OrderService(OrderRepository repo) { this.repo = repo; }
}

@Repository
public interface OrderRepository extends JpaRepository { }
```

Step 7: Common Mistakes and Misconceptions

Misconception: '@Service adds transaction support automatically.' False. @Transactional provides transactions, and it can be placed on any Spring-managed bean regardless of stereotype. @Service has zero runtime behavior of its own in most applications -- it is purely a documentation/architecture marker.

Mistake: Using @Component everywhere instead of proper stereotypes. This works technically but destroys readability. When a developer reads @Repository, they immediately understand: data access layer, expect DataAccessException, part of the persistence tier. @Component communicates nothing about architecture.

G

@Qualifier and @Primary

Resolving NoUniqueBeanDefinitionException -- disambiguation strategies

Step 1: Plain English -- Why Ambiguity Occurs

When you declare @Autowired on a field of type PaymentGateway, Spring searches its BeanFactory for all beans that implement PaymentGateway. If it finds exactly one: injection succeeds. If zero: NoSuchBeanDefinitionException. If two or more: **NoUniqueBeanDefinitionException**.

Two annotations solve this: **@Primary** marks one bean as the default winner. **@Qualifier** lets the injection site specify exactly which bean it wants by name or custom annotation value.

Step 2: Real-World Analogy

> Analogy: You ask a hotel front desk: 'I need a car.' Three rental companies are available. @Primary is the hotel's preferred partner -- ask generically, get that one. @Qualifier is saying: 'I specifically want the Tesla from GreenDrive Rentals' -- you name exactly what you want, overriding any defaults.

Step 3: Minimal Working Code

```
public interface PaymentGateway { void charge(double amount); }

@Component("stripeGateway")
@Primary   // wins when @Autowired by type without qualifier
public class StripeGateway implements PaymentGateway { ... }

@Component("paypalGateway")
public class PayPalGateway implements PaymentGateway { ... }

@Service
public class CheckoutService {
    private final PaymentGateway defaultGateway;  // injected: Stripe
    private final PaymentGateway paypalGateway;   // injected: PayPal

    public CheckoutService(
            PaymentGateway defaultGateway,
            @Qualifier("paypalGateway") PaymentGateway paypalGateway) {
        this.defaultGateway = defaultGateway;
        this.paypalGateway  = paypalGateway;
    }
}
```

Step 4: Diagram

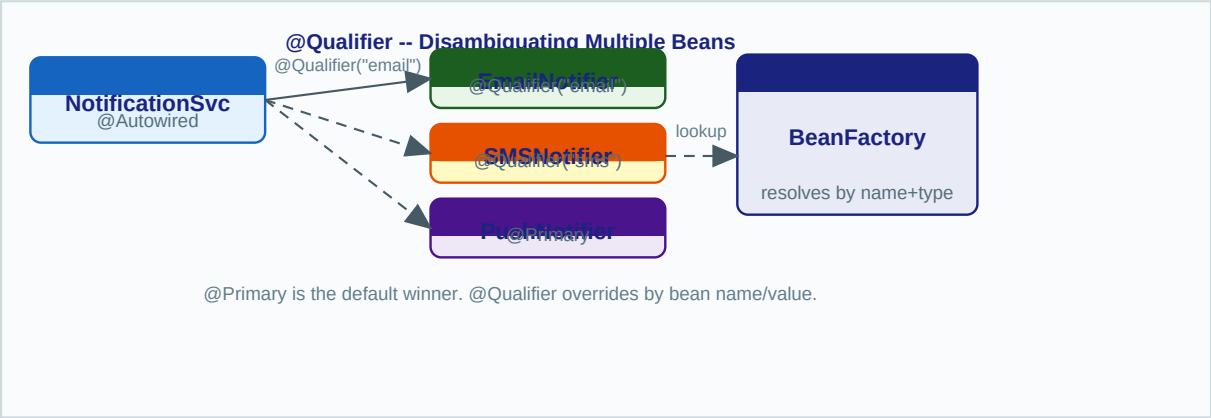

Fig G: @Primary is the default; @Qualifier overrides by explicit bean name

Step 5: Behind the Scenes -- Resolution Algorithm

Inside **DefaultListableBeanFactory.doResolveDependency()**:

* Find all candidate bean names whose type matches the required type (using isTypeMatch() which respects Java generics, e.g. List vs List).
* Filter by @Qualifier if present: look up the BeanDefinition for each candidate and check its registered qualifiers. @Qualifier value matches against bean name by default.
* If multiple candidates remain, check which has @Primary=true in its BeanDefinition.
* If two @Primary beans exist: throw NoUniqueBeanDefinitionException immediately.
* If exactly one remains: inject it. If zero remain: throw NoSuchBeanDefinitionException (unless required=false or the field has an Optional type).

Custom qualifiers: create your own qualifier annotation by meta-annotating with @Qualifier. This enables type-safe disambiguation: @PayPal instead of @Qualifier("paypalGateway") -- much harder to mistype, and refactor-safe in IDEs.

Step 6: Realistic Practical Example

```
// Custom qualifier annotation -- type-safe, no magic strings
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@Qualifier
public @interface FastCache { }

@Component @FastCache
public class RedisCache implements CacheService { ... }

@Component
public class DatabaseCache implements CacheService { ... }

@Service
public class ProductService {
    // Crystal clear: inject the Redis implementation specifically
    public ProductService(@FastCache CacheService cache) { ... }
}
```

Step 7: Common Mistakes and Misconceptions

> Mistake: @Qualifier with a bean name that does not match. Spring is case-sensitive: @Qualifier('redisCache') fails if the bean is registered as 'RedisCache'. The default naming is the class's simple name with first letter lowercased. Verify with context.getBeanDefinitionNames() if unsure.

Mistake: Putting @Primary on multiple beans of the same type. Spring throws NoUniqueBeanDefinitionException at startup: 'more than one primary bean found among candidates'. @Primary must mark exactly one bean per injectable type.

H

@Configuration and @Bean

ConfigurationClassPostProcessor, CGLIB proxying, programmatic bean definition

Step 1: Plain English -- What Is @Configuration?

@Configuration marks a class as a source of bean definitions. Inside it, methods annotated with @Bean tell Spring: 'this method returns an object that should be managed as a bean.' This is the Java-based alternative to XML configuration, offering full type safety and IDE support.

The critical distinction from a plain @Component class with @Bean methods: @Configuration classes are **CGLIB-proxied** by Spring. This proxy intercepts calls to @Bean methods and ensures singleton semantics -- the same instance is returned every time, even if you call the method directly from other @Bean methods.

Step 2: Real-World Analogy

> Analogy: @Configuration is a factory blueprint. @Bean methods are the assembly instructions for each product. The CGLIB proxy is a factory foreman who intercepts every production request: if Widget A was already made, the foreman returns the existing one from the warehouse. Without the foreman (no CGLIB), every request restarts fresh production -- you get a different widget each time.

Step 3: Minimal Working Code

```
@Configuration
public class DataConfig {
    // Spring calls dataSource() ONCE, caches the singleton.
    // When jdbcTemplate() calls dataSource() internally,
    // the CGLIB proxy returns the cached bean -- not a new one!
    @Bean
    public DataSource dataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:postgresql://localhost/shop");
        return ds;
    }

    @Bean
    public JdbcTemplate jdbcTemplate() {
        // dataSource() call intercepted by CGLIB -- returns singleton
        return new JdbcTemplate(dataSource());
    }
}
```

Step 4: Diagram

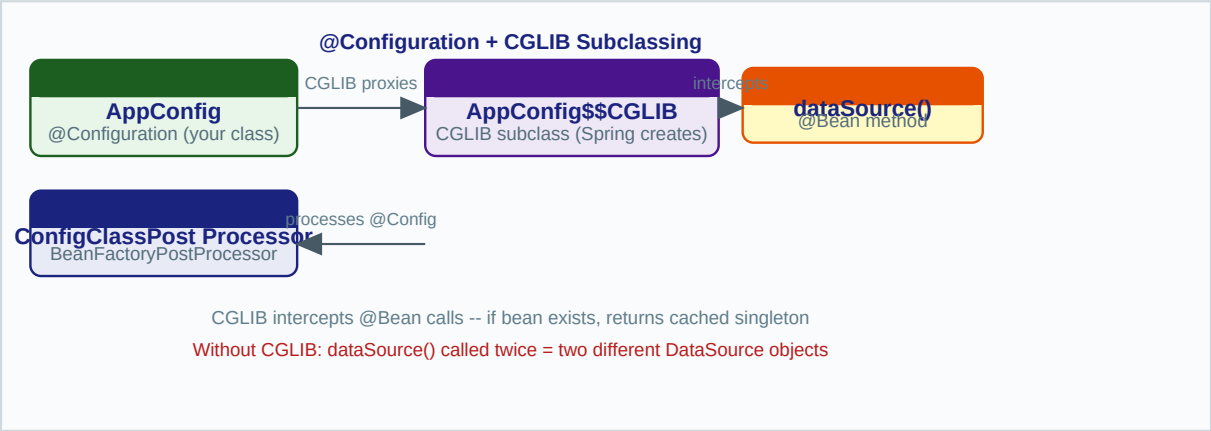

Fig H: ConfigurationClassPostProcessor creates CGLIB subclass; @Bean calls intercepted

Step 5: Behind the Scenes -- ConfigurationClassPostProcessor

ConfigurationClassPostProcessor (CCPP) is a **BeanFactoryPostProcessor** -- it runs BEFORE any beans are created, operating only on the BeanDefinition map.

* CCPP scans the BeanDefinitionRegistry for beans annotated with @Configuration (full mode) or @Component with @Bean methods (lite mode -- no CGLIB).

* For each @Configuration class, **ConfigurationClassParser** parses: @Bean methods, @Import, @ImportResource, @ComponentScan, @PropertySource. Each @Bean method generates a new BeanDefinition (method return type = bean type, method name = bean name).

* After parsing, CCPP uses **ConfigurationClassEnhancer** to generate a CGLIB subclass of every @Configuration class. The CGLIB MethodInterceptor overrides all @Bean methods.

* When a @Bean method is called at runtime, the interceptor first checks beanFactory.containsSingleton(beanName). If found: return existing instance. Otherwise: call super method (real Java code) and register the result.

* @Bean methods in @Component classes (lite mode) are NOT CGLIB-enhanced. Calling dataSource() from jdbcTemplate() in a lite-mode class creates a NEW DataSource object. This is a critical behavioral difference from full @Configuration mode.

Step 6: Realistic Practical Example

```
@Configuration
@PropertySource("classpath:db.properties")
public class AppConfig {
    @Value("${db.url}")
    private String dbUrl;

    @Bean @Primary
    public DataSource primaryDataSource() {
        return DataSourceBuilder.create().url(dbUrl).build();
    }

    @Bean("readOnly")
    public DataSource readOnlyDataSource() {
        return DataSourceBuilder.create()
            .url(dbUrl + "?readOnly=true").build();
    }

    @Bean
```

```
    public PlatformTransactionManager txManager() {
        // CGLIB ensures this is the same DataSource singleton
        return new DataSourceTransactionManager(primaryDataSource());
    }
}
```

Step 7: Common Mistakes and Misconceptions

Mistake: Using @Component instead of @Configuration when @Bean methods call each other. In a @Component class, @Bean methods are NOT proxied (lite mode). Each call creates a fresh instance, breaking singleton semantics silently. Always use @Configuration when @Bean methods reference each other.

Mistake: Making a @Configuration class final. CGLIB must subclass the configuration class. If it is final, Spring throws BeanDefinitionParsingException at startup: 'Cannot subclass final class'. Never use final on @Configuration classes.

Spring 6 / Spring Boot 3 note: proxyBeanMethods=false on @Configuration disables CGLIB proxying for performance when @Bean methods never call each other. This is common in Spring Boot auto-configuration classes and gives faster startup times.

Quick Reference -- Spring IoC and DI Fundamentals

Concept	Key Class / Annotation	When / Why to Use
IoC Container	ApplicationContext, DefaultListableBeanFactory	Replaces manual new. Manages lifecycle, scope, AOP.
BeanDefinition	ScannedGenericBeanDefinition, RootBeanDefinition	Internal recipe for creating a bean. Built before any bean is instantiated.
Constructor DI	@Autowired (optional in Spring 4.3+)	Preferred. Immutable fields (final), testable without Spring, fail-fast startup.
Setter DI	@Autowired on method	Optional deps. Can resolve circular deps. Two-phase init allows partial state.
Field DI	@Autowired on field	Concise but hides deps, hard to unit-test. Restrict to test classes.
@Component	ClassPathBeanDefinitionScanner	Generic bean. ASM bytecode scan finds it; BeanDefinition registered.
@Service	Meta-annotated @Component	Business logic layer marker. No extra Spring runtime behavior.
@Repository	PersistenceExceptionTranslationPostProcessor	DAO layer. Wraps persistence exceptions into DataAccessException hierarchy.
@Controller	DispatcherServlet, RequestMappingHandlerMapping	Web layer. Activates MVC request mapping. Use @RestController for APIs.
@Primary	BeanDefinition.isPrimary()	Default bean when multiple candidates match. At most one per injectable type.
@Qualifier	AutowiredAnnotationBeanPostProcessor	Precise selection by bean name or custom annotation. Overrides @Primary.
@Configuration	ConfigurationClassPostProcessor + CGLIB subclass	@Bean methods proxied for singleton semantics. Never make final.
@Bean	ConfigurationClassParser	Programmatic bean definition. Method name = bean name by default.

Injection Type Comparison

Criterion	Constructor Injection	Setter Injection	Field Injection

Immutability	Yes -- final fields possible	No	No
Required deps	Enforced at startup (NPE-proof)	Optional with required=false	Fails at runtime if missing
Unit testability	Just call constructor with mocks	Call setter in test setup	Need Mockito @InjectMocks or Spring context
Circular deps	Fails at startup (good!)	Spring resolves at population phase	Spring resolves at population phase
Verbosity	Medium -- explicit constructor	High -- separate setter methods	Low -- single annotation
Recommended?	Yes -- always preferred	Only for optional dependencies	Avoid in production code

Spring Internal Classes Reference

Class	Package	Role in IoC/DI
DefaultListableBeanFactory	beans.factory.support	Core IoC container: BeanDefinition registry and bean factory combined
AbstractAutowireCapableBeanFactory	beans.factory.support	Handles @Autowired wiring, populateBean, instantiateBean via reflection
AutowiredAnnotationBeanPostProcessor	beans.factory.annotation	Injects @Autowired fields and methods using Field.setAccessible + reflection
ClassPathBeanDefinitionScanner	context.annotation	Scans classpath with ASM; finds @Component classes; registers BeanDefinitions
ConfigurationClassPostProcessor	context.annotation	Parses @Configuration + @Bean + @Import; triggers CGLIB subclass generation
ConfigurationClassEnhancer	context.annotation	Generates CGLIB subclass for @Configuration -- intercepts @Bean method calls
BeanDefinitionRegistry	beans.factory.support	Interface: registerBeanDefinition(), getBeanDefinition(), containsBeanDefinition()
BeanDefinition	beans.factory.config	Stores all metadata about a bean before it is instantiated by the factory